ELI — Runtime Planning World

ELI v2.5.10

October 2026

Contents

ELI Runtime Surfaces, Planning, World, Tools & Plugins **1**
Runtime surfaces (`eli/runtime/`, the response/introspection group) 1
Planning / proactive (`eli/planning/`, 4.0k LOC) 2
World / autonomy model (`eli/world/`, 1.8k LOC) 3
Tools (`eli/tools/`, 7.6k LOC) 4

ELI Runtime Surfaces, Planning, World, Tools & Plugins

> **Updated 2026-09-30.** Stage 12 learning via `learning_coordinator.py`; goal autogenesis feeds the proactive stack. Task records gained an explicit resume path and a World bridge; habit learning is now gated against ELI's own automated actions.

The remaining subsystems: the `runtime/` response/introspection surfaces, the proactive planning layer, the "world"/autonomy model, and the tools + plugin system.

Runtime surfaces (`eli/runtime/`, the response/introspection group)

Beyond the grounding/evidence core (`grounding_and_evidence.md`), `runtime/` holds a large family of modules that render user-visible answers and introspect the live system:

- **Introspection**: `live_introspection.py` (`runtime_snapshot`, `last_trace`, `stored_user_name`, `mine_user_fact_candidates`, `build_report`, `agents_for_action`), `deterministic_introspection.py` (`classify_diagnostic_action → gather_evidence → format_evidence_block → maybe_handle` — deterministic diagnostic answers).
- **Response assembly**: `final_response_provider.py`, `response_contracts.py`, `response_policy.py`, `user_visible_response_surface.py`.
- **Personal memory surfaces**: `personal_memory_surface.py`, `personal_memory_clean_response.py`, `personal_memory_deep_response.py`, `profile_extractor.py`.
- **Status**: `frontier_status.py`, `reasoning_status.py`, `truth_report.py`, `reflection.py`.
- **Operator feed**: `operator_feed.py`, `operator_state.py`.

> **Over-fragmentation flag (the headline weakness for this group):** there are ~15 closely-related modules here doing "render a grounded user-visible answer / introspect runtime" with overlapping responsibilities (three `personal_memory_*` renderers alongside the response-contract, response-policy and user-visible-surface modules). This is the clearest "added beside, not folded in" cluster in the codebase and the prime consolidation target — it should collapse to a handful of well-named modules with clear ownership of "who produces the final string."

Planning / proactive (`eli/planning/`, 4.0k LOC)

Background cognition + goal/queue machinery: - `proactive_daemon.py` (1.0k) — the **self-improvement / proactive** loop. Uses a two-DB split: reads user-facing memory/conversation from the **user DB**, writes proactive observations/improvements/errors to the **agent DB** (keeps ELI's own machinations out of user recall). Background thread. - `autonomy_scheduler.py` — schedules autonomous actions (throttled). Also the "decide to invoke it" layer for self-upgrade and LoRA retraining (`_maybe_check_self_upgrade`/`_maybe_check_lora_retrain`, added 2026-09-18): both mechanisms were fully built and safety-gated but pull-only — nothing inside ELI ever decided on its own to use them, even with a newer release on GitHub or enough reviewed training data sitting idle. Only fires in `goal_driven` operator-policy mode (every other mode is unchanged, still pull-only), on a multi-day throttle per check kind, and routes through the same `schedule_request(kind=...)` entry point and downstream safety machinery a human-typed "update ELI overnight" already used — this closes the trigger gap without adding a new execution path. - `goal_store.py`, `proposal_queue.py`, `proposal_adapters.py`, `attention_queue.py`, `jobqueue.py`, `operator_goal_actions.py`, `habits.py` — goal persistence, capability-proposal queue, attention prioritisation, a job queue, and habit scheduling. - **Routine detection** (`habits.py`, `routine_stats.py`): app launches are clustered on the clock as a circle (neighbours within 30 minutes join, across midnight too), so 08:58 and 09:02 are one routine. A routine needs launches on at least three distinct days. `detect_shift` flags a routine whose recent times sit an hour or more from the earlier ones, and `detect_lapse` flags one that stopped, judged against the days the person was active at all; each is recorded once as an observation and never changes a rule you enabled. `detect_habits` returns `{suggested, shifts, lapsed}`. The scheduler keeps its run keys on disk so a restart in the same minute does not run a habit twice, and a plain-text reply from the engine is reported as unverified rather than as success. - **Lessons** (`runtime/lessons.py`): failures that recur (same action, same error, three or more times) become lessons with a trigger, evidence, proposed change, predicted outcome and an expiry. Each finished action of that kind gives its lessons one check; a lesson is retired when it expires unconfirmed or its checks show it does not help, and renewed a few times when they do. - **Lessons, checked against the baseline.** A lesson records the action's success rate before it existed and is retired unless later runs beat that rate by a margin. It is scoped to the argument names all its failures shared and is checked on sibling actions with the same failure category, so `transfers()` says whether the advice held elsewhere. - **Reliability and calibration** (`evidence_ledger.py`): `action_reliability` and `predict_success` give a recency-weighted success rate per action from the recorded tool runs; each run stores the prediction made before it, and `calibration_report` scores them (Brier score and predicted versus observed by band). The awareness block lists actions that have been failing lately. - **Task records** (`goal_store.py`): a task is a goal the scheduler does not tick, carrying constraints, decisions, finished steps, artifacts and open questions. `capture_task_events` reads clear phrasings from the user's messages ("we decided to", "we still need to", "let's work on"), and `task_brief` puts unfinished work into the next session's awareness block — but only within a 14-day idle window. `find_task(query, include_idle=True)` (2026-09-29) bypasses that window on an explicit ask ("continue the RAIMS project" routes to `RESUME_TASK`, a real router/executor action — not a chat-routed guess), and states real staleness ("last touched N days ago") rather than silently implying continuity. `close_task(goal_id)` is the explicit, non-inferred completion path. Opening/closing a task also notifies the World system (below) — a task is now a real, visible object in Eli's World, not just a database row. - **Habit-learning integrity** (2026-09-30): `_learn_from_result`'s writes into `habit_events` — the table `detect_habits` mines for recurring routines — are gated on whether the triggering call actually came from the user. `habits_scheduler.py` firing an approved rule calls `engine.process(source="habit")`; without the gate, that automated firing would land back in `habit_events` and let ELI's own automation reinforce itself as if it were a fresh, independent observation of a routine. `_is_autonomous_source()` (shared with `_settle_recall_outcome`, which already checked this) treats `habit`/`proactive`/`scheduler`/`system`/`autonomy` as non-user origins. - **World events.** `review_completed` eases repair pressure after any review; `repair_completed` fires only for a verified repair. - `goal_autogenesis.py` (NEW, 2026-06-08) — **closes the autonomy loop**: the goal stack (goal_store → `due_goals` → `governed_goal_tick` → proposal_queue) was fully wired but the store was always EMPTY because `create_goal` was operator-only, so the scheduler ticked nothing and ELI only reacted. `propose_goals_from_signals` turns the signals ELI already computes each proactive tick — world-model awareness suggestions (≥0.6), recurring failures (≥5×), and code-health

improvements (oversized fns / duplicate blocks in his own god-files) — into governed goals. Each is `autonomy_mode="proposal_only"` (surfaces for approval; never silent execution), deduped by a stable signal-derived `goal_id` (idempotent every tick), capped at `MAX_AUTO_GOALS=6`, logged as a `self_goal` observation. Wired into `proactive_daemon` where the world-suggestions + patterns + improvements are in scope. His agenda now spans failure-repair, world-driven, AND self-improvement — i.e. he sets his own intentions, governed.

World / autonomy model (`eli/world/`, 1.8k LOC)

The substrate behind ELI's emergent self-state (autonomy pressure, awareness, "anomaly room" — intended behaviour, see memory `eli-emergent-voice`): - `agency/autonomy_engine.py` — `EliWorldAutonomyEngine`: ingests world events, maintains awareness/autonomy state. Every proposed `WorldAction` is `policy.classify()`'d, provenance-logged (`record_provenance`: actor, triggering event, full action, awareness snapshot — logged whether or not it's allowed), and only actually applied if `policy.allowed()`; a blocked or approval-required action is recorded but not executed. `_apply_action` handles `CREATE_OBJECT` and (2026-09-29) `RETIRE_OBJECT` — previously declared in the schema/policy with zero handler, a real gap since `RETIRE_OBJECT` is the only way a `WorldObject.retired` flag ever gets set. Opening a task creates a per-task workbench object (`_create_task_object_action`, linked via `WorldObject.links` so it's distinct from the shared template+room-keyed object every other `project_workbench` trigger reuses — retiring one task must not retire every task's marker); closing a task retires it. Internal reasoning stages are spatially visualised: `_REASONING_STAGE_ROOM_MAP` routes each `(reasoning_mode, stage_name)` pair (chain-of-thought's scratchpad vs. final synthesis, tree-of-thoughts' branch proposal vs. development, etc.) to a specific room (Reflection Chamber, Simulation Lab, Evidence Wall, Anomaly Room) and materialises stage-specific objects there; `propose_actions()` also reads live awareness thresholds (`memory_confidence < 0.45` spawns a diagnostic-bench object, `reflection_depth > 0.55` spawns a reflection lectern) — the world externalises ELI's own cognitive state as spatial objects as it happens, not just a chat avatar. - `agency/policy.py` — `EliWorldPolicy`: keyword-based action classifier (`BLOCKED_KEYWORDS`, `APPROVAL_KEYWORDS`) plus an enumerated safe-action allowlist; default-deny (unmatched action types fall to `APPROVAL_REQUIRED`, not silently allowed). **Not the same object as `agency/world_constitution.py`'s declared `WORLD_CONSTITUTION` tiers** (`free_actions/approval_required/blocked`) — that dict is only ever read by `get_world_constitution()` to stamp metadata into the persisted world-state JSON (`persistence/storage.py`); editing it changes what's *displayed*, not what's *enforced*. The two lists currently agree in intent but are hand-maintained separately and can drift. - `agency/goal_ecology.py` — `decay_goals()`: goal urgency decays by a fixed 0.02 per ecology tick (called from every `load()`/`ingest_event()`), abandoned goals pruned. Lightweight; the richer goal-generation logic lives in `planning/goal_autogenesis.py` above, not here. - `local_world_bridge.py` — `append_event`, `get_world_state`, `get_awareness_driven_suggestions` (**throttled to once/hour to prevent auto-trigger loops**). Its `priority >= 0.70` consumer in `proactive_daemon.py` (`_handle_high_priority_world_suggestion`, 2026-09-30) defers to an active user turn and reuses `attention_queue.append_attention`'s own suppression window (30 min) as its cooldown, rather than acting with no interruption-cost check at all. It also no longer claims `"[AUTO] ... triggered"` — nothing in this path calls the executor, so the surfaced text and stored memory note now say "suggested." - `world_event_bus.py` — `fire_confidence_event(...)` etc.; the engine feeds bus-dispatch confidence here (non-blocking world-awareness feed). - `core/schemas.py` — `WorldEvent`, `AwarenessState`. - `renderers/pyside6/world_scene.py` + `world_panel.py` — a **visual** "ELI's World" panel. - `persistence/storage.py` — world-state persistence. Three properties were added after live faults (2026-08-09/10) and are load-bearing: * **Absolute paths.** `world_dir()` resolves through `get_paths()`. The journal, provenance ledger and snapshot modules previously kept relative `Path("artifacts/world/...")` constants and wrote wherever ELI was launched from — a different place from ~ than from the project directory, and in a packaged build a failed write to `/artifacts`. * **Schema-tolerant load.** `_fit()` builds each dataclass from the saved dict while ignoring fields it no longer declares. Before it, one unknown key made `AwarenessState(**data)` raise, `load()` caught it, filed the state as `.corrupt_*` and returned a fresh default world — so a single renamed field would silently wipe the user's rooms, objects, goals and habits on upgrade. Two of the five corrupt backups on the dev machine parse as valid JSON; they were condemned by exactly this and nothing else. * **Bounded logs.** `actions.jsonl` reached 41 MB / 80,576

lines because nothing rotated it. `_trim_jsonl` keeps the newest 20,000 lines (counted, not size-capped, so a trim never splits a JSON record) and checks on the first append of each process plus every 250 thereafter — the per-process counter alone meant a short session never trimmed at all.

Tools (`eli/tools/`, 7.6k LOC)

- `image_engine/` (the package under `eli/tools/image_engine/image_engine/`: `service.py`, `visual_core.py`, `plotting.py`, `prompt_compiler.py`, `quality.py`, `project_analyzer.py`, `memory.py`): Pillow/numpy base image generation with an optional diffusers path, the bundled fallback.
- **Weak / watch:**
  1. **`runtime/` over-fragmentation** — ~15 overlapping response/introspection modules. Highest-value consolidation in the project after the god-files.
  2. **Image-engine duplication — RESOLVED** — the shadowed standalone module was deleted; only the `tools/image_engine/` package remains.
  3. The planning queues (`goal_store`/`proposal_queue`/`attention_queue`/ `jobqueue`) overlap conceptually and could likely unify into one work-queue abstraction.
  4. Plugin registry points at an `eli-plugins/registry` GitHub URL — fine as opt-in, but ensure it degrades cleanly offline (the bundled fallback exists; verify it's always used when the network is absent).
  5. **Self-upgrade/LoRA were pull-only — RESOLVED 2026-09-18** — `autonomy_scheduler.py` now decides on its own, in `goal_driven` mode, when to schedule a `self_upgrade` check or a `lora` retrain, instead of requiring an explicit human ask or pre-schedule every time.